

Figure 5: VisualVM monitoring sampler CPU snapshot

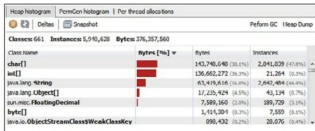


Figure 6: VisualVM monitoring sampler memory heap histogram

Snapshot

A snapshot of the running application's CPU utilisation can be obtained by clicking the *Snapshot* button. In the snapshot view, a drill down of the time spent in each method can be obtained.

As can be seen from the screen shot, the full call tree of the *main()* method can be drilled down to identify the times spent in each sub-method. In the above example, it can be seen that *intern()* in *String* is the method that takes a significant amount of time. In addition to the call tree, the method hotspots can be viewed by clicking the *Hot Spots* tab. Instead of a method view, a class or package level view also can be obtained. During identification of bottlenecks, the method view is most useful.

Similar to CPU sampling, memory sampling also can be performed to identify the largest memory consumers by bytes occupied or the number of instances. Memory allocation in each memory area – heap or PermGen – can be monitored. A per thread allocation of memory can also be viewed. Like CPU sampling, memory occupied by specific classes or packages can be viewed by setting an appropriate filter.

Snapshots can be taken at various intervals. However, unlike CPU sampling, a drill down cannot be performed on memory snapshots. Snapshots taken at multiple instances of time can be compared to view the differences in the memory allocation of those objects. As in the case of viewing memory samples, the snapshot difference can also be filtered by a specific package or class. This feature is very useful when a problematic class is identified and specifics about the amount of space taken by that object are needed.

Profiler

Profiler's user interface is very similar to the Sampler user interface. It can be invoked by clicking the *Profiler* tab and choosing *CPU* or *Memory*. Once this choice is made, a confirmation that byte code modification for instrumentation and profiling is taking place can be seen on the Java command line.

Depending on the code base, CPU utilisation on the machine showing profiling results could take a significant amount of time since the JVM needs to instrument the classes, reload the classes in JVM and start the profiling.

Visual GC plug-in

A very useful plug-in to visually examine the different memory areas within the heap region is the VisualGC plug-in. This can be by downloaded navigating to Tools –> Options --> Available Plugins and choosing Visual GC.

Visual GC shows the same information as verbose GC, but in a more user-friendly way. In addition to the different memory areas, it also shows information on the Just-In-Time (JIT) compilation. The different memory areas can be tuned based on information from Visual GC. For instance, if Eden space is getting filled up frequently, but the Old Generation is underutilised, the size of Eden can be adjusted using the *NewRatio* parameter on the JVM command line. 

References

- [1] Eclipse Memory Analysis Tool (MAT) www.eclipse.org/mat
- [2] Object Query Language - http://en.wikipedia.org/wiki/Object_Query_Language
- [3] IBM Heap Dumps http://publib.boulder.ibm.com/infocenter/ieduasst/v1r1m0/index.jsp?topic=/com.ibm.ias.was_v6/was/6.0.1/PDF/WASv601_zOS_Heapdumps/player.html
- [4] HPFROF: A Heap/CPU Profiling Tool - <http://docs.oracle.com/javase/7/docs/technotes/samples/hprof.html>
- [5] Path to GC Queries: http://pic.dhe.ibm.com/infocenter/isa/v4r1m0/index.jsp?topic=/%2Fcom.ibm.java.diagnostics.memory.analyzer.doc%2Fpath_to_gcroots.html
- [6] Java Tools
 - Jps: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jps.html>
 - Jstat: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jstat.html>
 - JMap: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jmap.html>
 - JHat: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jhat.html>
- [7] Java Instrumentation package: <http://docs.oracle.com/javase/7/docs/api/index.html?java/lang/instrument/package-summary.html>
- [8] Visual GC help: <http://www.oracle.com/technetwork/java/visualgc-136680.html>
- [9] IBM Developer Works Community: <https://www.ibm.com/developerworks/community/>
- [10] Java HPFROF: <http://docs.oracle.com/javase/7/docs/technotes/samples/hprof.html>
- [11] Portable Heap Dump: http://pic.dhe.ibm.com/infocenter/java7sdk/v7r0/index.jsp?topic=/%2Fcom.ibm.java.win.70.doc%2Fdiag%2Ftools%2Fheapdump_phd_format.html

By: Murali Suraparaju

The author is an MTech in Computer Science who has worked extensively on building enterprise applications using JEE technologies, and has developed solutions for enterprise and embedded products. He is a member of the Performance Engineering practice in the Financial Services division of Infosys Limited.